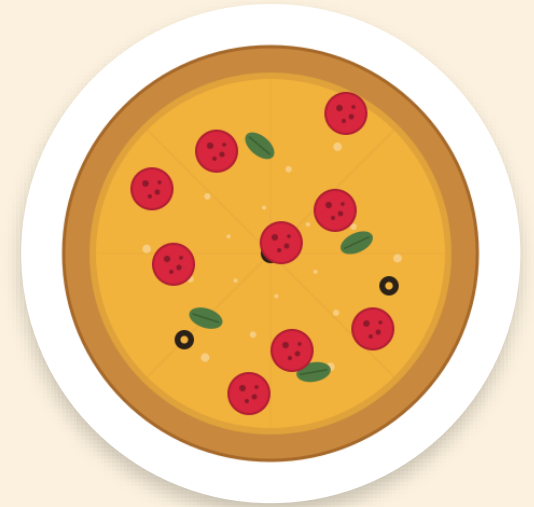


PIZZA SALES SQL PROJECT

Business insights from a pizza restaurant's order data — one query at a time.

By Ayush Kumar Singh

BASIC • INTERMEDIATE • ADVANCED



ABOUT THE PROJECT

This project uses SQL to analyze real-world pizza sales data — covering orders, revenue, pizza types, and customer ordering patterns. The goal is to turn raw transactional data into clear business insights using queries ranging from simple counts to advanced window functions.

4

Database
Tables

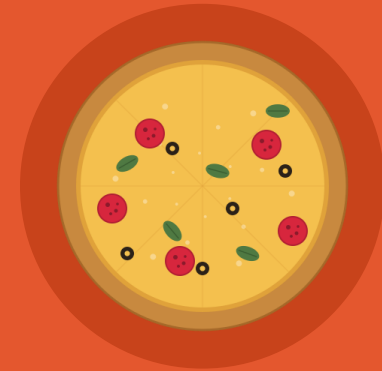
13

SQL
Queries

3

Difficulty
Levels

Built by Ayush Kumar Singh



Dataset

orders • order_details
pizzas • pizza_types

DATABASE SCHEMA

Four related tables power every query in this project.



orders

order_id
date
time



order_details

order_details_id
order_id
pizza_id
quantity



pizzas

pizza_id
pizza_type_id
size
price



pizza_types

pizza_type_id
name
category
ingredients

BASIC QUERIES

1

Retrieve the total number of orders placed.

2

Calculate the total revenue generated from pizza sales.

3

Identify the highest-priced pizza.

4

Identify the most common pizza size ordered.

5

List the top 5 most ordered pizza types along with their quantities.

Q1. Retrieve the total number of orders placed.

```
SELECT COUNT(order_id) AS total_orders  
FROM orders;
```

Explanation

We simply count every row in the orders table, since each row represents one order placed.

Q2. Calculate the total revenue generated from pizza sales.

```
SELECT ROUND(SUM(od.quantity * p.price), 2) AS total_revenue  
FROM order_details od  
JOIN pizzas p ON od.pizza_id = p.pizza_id;
```

Explanation

Revenue per order line is quantity multiplied by price. Joining order_details to pizzas pulls in the price for each line item, and summing across every row gives the restaurant's total revenue.

Q3. Identify the highest-priced pizza.

```
SELECT pt.name, p.price
FROM pizzas p
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
ORDER BY p.price DESC
LIMIT 1;
```

Explanation

We join pizzas to pizza_types to get readable names, sort by price in descending order, and take the top row to find the single most expensive pizza.

Q4. Identify the most common pizza size ordered.

```
SELECT p.size, COUNT(od.order_details_id) AS order_count
FROM pizzas p
JOIN order_details od ON p.pizza_id = od.pizza_id
GROUP BY p.size
ORDER BY order_count DESC
LIMIT 1;
```

Explanation

We group all order line items by pizza size, count how many times each size appears, and return the size with the highest count.

Q5. List the top 5 most ordered pizza types with quantities.

```
SELECT pt.name, SUM(od.quantity) AS total_quantity
FROM pizza_types pt
JOIN pizzas p ON pt.pizza_type_id = p.pizza_type_id
JOIN order_details od ON p.pizza_id = od.pizza_id
GROUP BY pt.name
ORDER BY total_quantity DESC
LIMIT 5;
```

Explanation

We chain joins from `pizza_types` through `pizzas` to `order_details` so each line item can be tied back to a pizza name, sum the quantities per name, and keep the top 5.

INTERMEDIATE QUERIES

1

Find total quantity ordered per pizza category.

2

Determine the distribution of orders by hour of the day.

3

Find the category-wise distribution of pizzas.

4

Calculate the average number of pizzas ordered per day.

5

Determine the top 3 most ordered pizza types by revenue.

Q6. Find the total quantity of each pizza category ordered.

```
SELECT pt.category, SUM(od.quantity) AS total_quantity
FROM pizza_types pt
JOIN pizzas p ON pt.pizza_type_id = p.pizza_type_id
JOIN order_details od ON p.pizza_id = od.pizza_id
GROUP BY pt.category
ORDER BY total_quantity DESC;
```

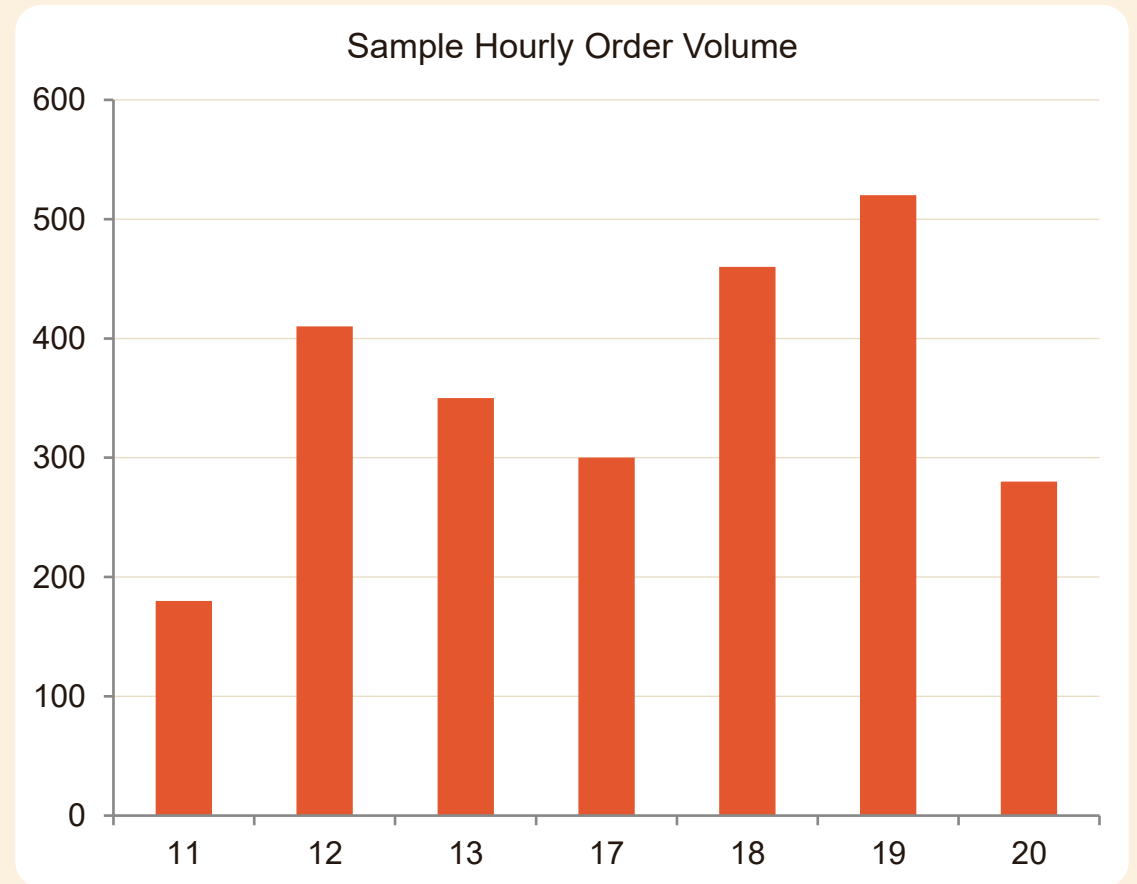
Explanation

Same join path as the basic queries, but this time we group by category (Classic, Veggie, Supreme...) instead of individual pizza name, giving quantity totals at the category level.

Q7. Determine the distribution of orders by hour of the day.

```
SELECT HOUR(o.time) AS order_hour,  
       COUNT(o.order_id) AS order_count  
FROM orders o  
GROUP BY HOUR(o.time)  
ORDER BY order_hour;
```

Grouping by the HOUR() of each order's timestamp reveals peak ordering windows — typically lunch and dinner rushes — helping with staffing and prep decisions.



Q8. Find the category-wise distribution of pizzas.

```
SELECT category, COUNT(name) AS pizza_count  
FROM pizza_types  
GROUP BY category;
```

Explanation

This counts how many distinct pizza types exist within each category — the menu composition — as opposed to Q6, which measured how many were actually ordered.

Q9. Group orders by date and average pizzas ordered per day.

```
SELECT ROUND(AVG(pizzas_per_day), 0) AS avg_pizzas_per_day
FROM (
    SELECT o.date, SUM(od.quantity) AS pizzas_per_day
    FROM orders o
    JOIN order_details od ON o.order_id = od.order_id
    GROUP BY o.date
) AS daily_totals;
```

Explanation

The inner query first computes total pizzas sold per calendar date. The outer query then averages those daily totals to get one overall average-per-day figure.

Q10. Determine the top 3 most ordered pizza types by revenue.

```
SELECT pt.name, SUM(od.quantity * p.price) AS revenue
FROM pizza_types pt
JOIN pizzas p ON pt.pizza_type_id = p.pizza_type_id
JOIN order_details od ON p.pizza_id = od.pizza_id
GROUP BY pt.name
ORDER BY revenue DESC
LIMIT 3;
```

Explanation

Unlike Q5, which ranked by quantity, here we rank by revenue — quantity multiplied by price, summed per pizza name — to find which types bring in the most money, not just the most orders.

ADVANCED QUERIES

1

Calculate each pizza type's % contribution to total revenue.

2

Analyze cumulative revenue generated over time.

3

Find the top 3 pizza types by revenue, within each category.

Q11. Calculate each pizza type's % contribution to total revenue.

```
SELECT pt.name,  
       ROUND(SUM(od.quantity * p.price) /  
             (SELECT SUM(od2.quantity * p2.price)  
               FROM order_details od2  
               JOIN pizzas p2 ON od2.pizza_id = p2.pizza_id) * 100,  
             2) AS revenue_percentage  
FROM pizza_types pt  
JOIN pizzas p ON pt.pizza_type_id = p.pizza_type_id  
JOIN order_details od ON p.pizza_id = od.pizza_id  
GROUP BY pt.name  
ORDER BY revenue_percentage DESC;
```

Explanation

A scalar subquery first calculates total revenue across the whole business. Each pizza type's own revenue is then divided by that grand total and multiplied by 100 to express it as a percentage share.

Q12. Analyze the cumulative revenue generated over time.

```
SELECT date, revenue,  
       SUM(revenue) OVER (ORDER BY date) AS cumulative_revenue  
FROM (  
  SELECT o.date, SUM(od.quantity * p.price) AS revenue  
  FROM orders o  
  JOIN order_details od ON o.order_id = od.order_id  
  JOIN pizzas p ON od.pizza_id = p.pizza_id  
  GROUP BY o.date  
) AS daily_revenue;
```

Explanation

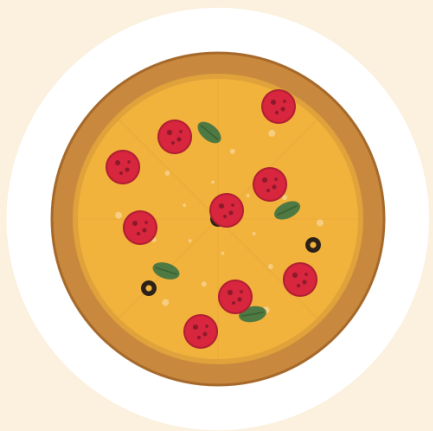
The subquery computes total revenue per day. The window function `SUM(...) OVER (ORDER BY date)` then adds up revenue running forward through time, giving a running cumulative total.

Q13. Top 3 most ordered pizza types by revenue, per category.

```
SELECT category, name, revenue FROM (  
  SELECT pt.category, pt.name,  
         SUM(od.quantity*p.price) AS revenue,  
         RANK() OVER (  
           PARTITION BY pt.category  
           ORDER BY SUM(od.quantity*p.price) DESC  
         ) AS rnk  
  FROM pizza_types pt  
  JOIN pizzas p ON pt.pizza_type_id=p.pizza_type_id  
  JOIN order_details od ON p.pizza_id=od.pizza_id  
  GROUP BY pt.category, pt.name  
) ranked WHERE rnk <= 3;
```



RANK() OVER (PARTITION BY category ...) ranks pizza types by revenue separately within each category. Filtering `rnk <= 3` keeps only the top 3 earners per category — surfacing each category's best sellers.



THANK YOU

Questions about the queries? Let's dig into the data together.

Ayush Kumar Singh